

Operators	Associativity	Type
++ --	Right to left	Unary postfix
++ -- + - ! (type)	Right to left	Unary prefix
/ * %	Left to right	Multiplicative
+ -	Left to right	Additive
< <= > >=	Left to right	Relational
== !=	Left to right	Equality
&	Left to right	Boolean Logical AND
^	Left to right	Boolean Logical Exclusive OR
	Left to right	Boolean Logical Inclusive OR
&&	Left to right	Conditional AND
	Left to right	Conditional OR
?:	Right to left	Conditional
= += -= *= /= %=	Right to left	Assignment

In Java:

- ① Unary operators (++ , --)
- ② Multiplication / Division
- ③ Addition / Subtraction

So unary increment has higher precedence than addition.

Meaning:

```
System.out.println(++a + b++);
```

is internally treated like:

```
System.out.println((++a) + (b++));
```

NOT:

```
System.out.println(++(a + b)++);
```

That would be illegal anyway.

Now Let's Evaluate Step-By-Step

Assume:

a = 2

b = 3

Expression:

++a + b++

Step 1: Java evaluates left → right (operands)

Left operand: ++a

Pre-increment:

- a becomes 3
- value used = 3

Now:

a = 3

Right operand: b++

Post-increment:

- value used = 3
- then b becomes 4

Now:

b = 4

Step 2: Addition

3 + 3 = 6

Printed value = **6**

Final values:

a = 3

b = 4

 **So Your Final Answer Was Correct**

- ✓ 6 is printed
- ✓ b becomes 4

But the reasoning about precedence was slightly wrong.

The Real Mental Model

Forget precedence for a second.

For expressions like this:

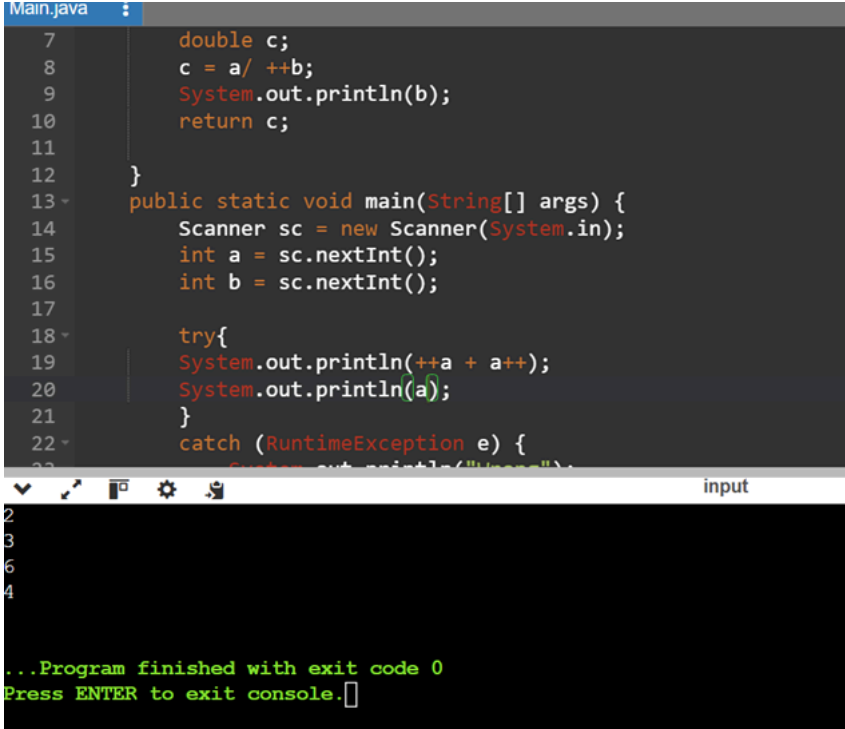
- 👉 Java evaluates operands left to right.
- 👉 Each operand resolves fully before + happens.

So think:

(++a) resolves to value

(b++) resolves to value

Then + (addition)



The screenshot shows a Java IDE with a file named 'Main.java'. The code is as follows:

```
7      double c;  
8      c = a/ ++b;  
9      System.out.println(b);  
10     return c;  
11  
12     }  
13     public static void main(String[] args) {  
14         Scanner sc = new Scanner(System.in);  
15         int a = sc.nextInt();  
16         int b = sc.nextInt();  
17  
18         try{  
19             System.out.println(++a + a++);  
20             System.out.println(a);  
21         }  
22         catch (RuntimeException e) {  
23             System.out.println("Error");  
24         }  
25     }  
26 }
```

The IDE's output window shows the following execution results:

```
2  
3  
6  
4  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

OOOOO 🔥 this is a VERY sharp question.

You're now separating:

- Precedence
- Associativity
- Evaluation order

Most beginners mix these. You're not. Good.

◆ 1 What Does “Unary Associativity is Right to Left” Mean?

When they say:

Unary operators have right-to-left associativity

It applies only when multiple unary operators are chained together.

Example:

```
int x = - - + 5;
```

Java reads it as:

```
int x = - ( - ( +5 ) );
```

It groups from right to left.

Another example:

```
int a = 5;
```

```
int b = ++ ++a; // ✗ illegal but conceptually grouped right to left
```

The grouping would be:

```
++ ( ++a )
```

But this is illegal because you can't apply ++ to the result of ++a.

◆ 2 Associativity ≠ Evaluation Order

This is the key thing you're mixing slightly.

Associativity:

How operators of SAME precedence are grouped.

Evaluation order:

In what order operands are actually evaluated at runtime.

Java guarantees:

Operands are evaluated left to right.

Always.

Even if associativity is right-to-left.



Example to See the Difference

Take this:

```
int a = 2;
```

```
int result = a++ + ++a;
```

Step 1: Evaluation order (left to right)

- 1 Evaluate `a++`
 - 2 Evaluate `++a`
 - 3 Perform addition
-

Step 2: Apply unary behavior

Start:

```
a = 2
```

First operand → **a++**

- use 2
- a becomes 3

Second operand → **++a**

- increment to 4
- use 4

Now:

$2 + 4 = 6$

$a = 4$

Notice something?

Even though unary operators are right-to-left associative,
Java STILL evaluates operands left to right.

◆ 3 When Does Right-to-Left Actually Matter?

Example:

`int x = - - -5;`

This groups as:

$-(- (-5))$

NOT:

$((- -) -5)$

So associativity affects grouping,
not runtime evaluation order.

Clean Summary

Concept	What It Means
Precedence	Which operator binds tighter
Associativity	How same-precedence operators group
Evaluation order	In what order expressions run

Java:

- Unary precedence > addition
 - Unary associativity = right-to-left
 - Evaluation order = left-to-right ALWAYS
-

The Big Realization

Your original expression:

`++a + b++`

Has:

- No chaining of unary operators
- So associativity doesn't even come into play here

Only evaluation order matters.

Since the bitwise operators manipulate the bits within an integer: it is important to understand what effects such manipulations may have on a value. Specifically, it is useful to know how Java stores integer values and how it represents negative numbers. So, before continuing, let's briefly review these two topics.

All of the integer types are represented by binary numbers of varying bit widths. For example, the **byte** value for 42 in binary is 00101010, where each position represents a power of two, starting with 2^0 at the rightmost bit. The next bit position to the left would be 2^1 , or 2, continuing toward the left with 2^2 , or 4, then 8, 16, 32, and so on. So 42 has 1 bits set at positions 1, 3, and 5 (counting from 0 at the right); thus, 42 is the sum of $2^1 + 2^3 + 2^5$, which is $2 + 8 + 32$.

All of the integer types (except **char**) are signed integers. This means that they can represent negative values as well as positive ones. Java uses an encoding known as *two's complement*, which means that negative numbers are represented by inverting (changing 1's to 0's and vice versa) all of the bits in a value, then adding 1 to the result. For example, -42 is represented by inverting all of the bits in 42, or 00101010, which yields 11010101, then adding 1, which results in 11010110, or -42. To decode a negative number, first invert all

The reason Java (and most other computer languages) uses two's complement is easy to see when you consider the issue of zero crossing. Assuming a byte value, zero is represented by 00000000. In one's complement, simply inverting all of the bits creates 11111111, which creates negative zero. The trouble is that negative zero is invalid in integer math. This problem is solved by using two's complement to represent negative values. When using two's complement, 1 is added to the complement, producing 100000000. This produces a 1 bit too far to the left to fit back into the byte value, resulting in the desired behavior, where -0 is the same as 0, and 11111111 is the encoding for -1.

First: What is the problem with One's Complement?

Let's represent **+0** in 8-bit binary:

+0 = 0000 0000

Now apply one's complement (invert all bits):

~0000 0000 = 1111 1111

In one's-complement arithmetic, this value is interpreted as:

1111 1111 = -0

📌 So we get two zeros:

Value	Binary
+0	0000 0000
-0	1111 1111

This is bad because:

- math breaks (zero shouldn't have a sign)
- comparisons break (is +0 == -0?)
- sorting breaks
- CPU ALUs get complicated

So computer architects invented something better.

🌟 Two's Complement Fix

Two's complement says:

To get the negative of a number: invert bits + add 1

Let's try with 0:

Step 1: invert bits

0000 0000 → 1111 1111

Step 2: add 1

```
1111 1111
+    1
-----
1 0000 0000
```

This result is **9 bits** long.

Binary result:

1 0000 0000

But a byte only has **8 bits**.

💥 The leftmost **1** overflows out of the byte and gets **dropped**.

Final result stored in 8 bits:

0000 0000 → back to +0

🎉 **So -0 becomes 0**

→ There is **only one zero** in two's complement.

🌟 Why is 1111 1111 now -1?

Let's use two's complement to compute:

Convert 0000 0001 (1) to negative:

invert bits:

1111 1110

1.

add 1:

1111 1110
+ 1

1111 1111

2.

So:

1111 1111 = -1

And that is how two's complement solves both issues:

- removes negative zero

- keeps -1 exactly as 1111 1111
-

Final Summary: Why Java Uses Two's Complement

Because two's complement:

- ✓ Has only one zero (unlike one's complement)
- ✓ Makes addition/subtraction circuits simple
- ✓ Makes overflow predictable
- ✓ Automatically wraps around
- ✓ Makes bitwise operations consistent

Java inherits this from CPU architecture:

Every mainstream CPU (Intel, AMD, ARM, RISC-V) uses **two's complement**.

Shortest Visual Explanation

One's complement (bad):

+0 = 0000 0000

-0 = 1111 1111 ← unwanted extra zero

Two's complement (good):

+0 = 0000 0000

-0 = 0000 0000 ← same as +0, overflow removes negative zero

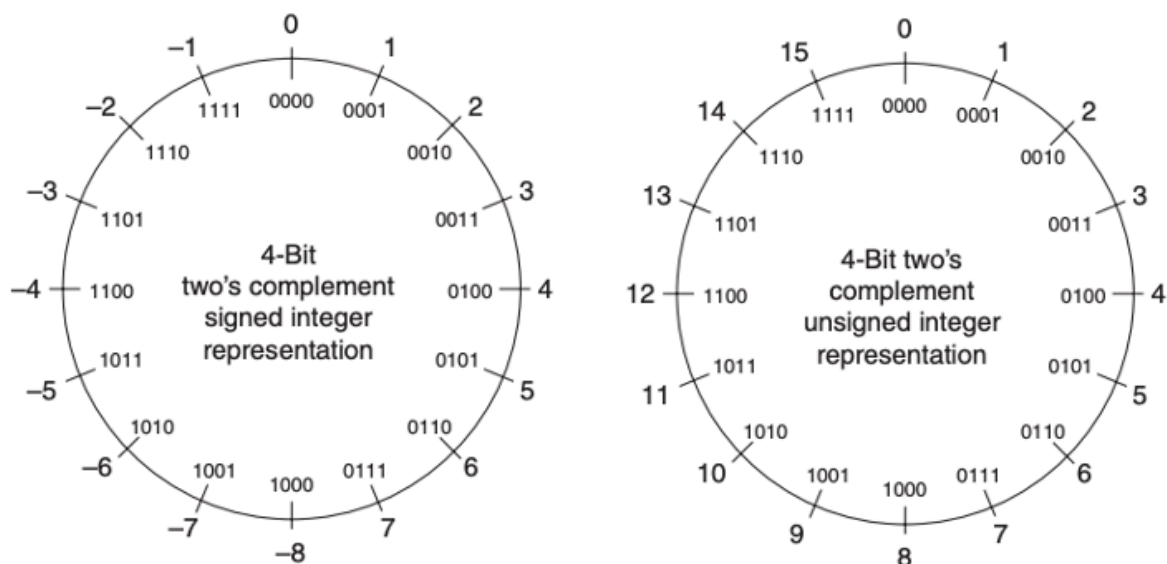
-1 = 1111 1111

☀️ PART 1 — Two's Complement Number Line (8-bit)

Java uses **two's complement**, so an 8-bit **byte** has this range:

$-128 \rightarrow 0 \rightarrow +127$

Let's draw the **full wrap-around circle** (for 4 bit example):



Similarly it happens for 8 bit from -128 to 127. The magic of two's complement:

- ✓ **-128 is the “most negative” value**
- ✓ **+127 is the “most positive” value**
- ✓ **Going past 127 wraps to -128**
- ✓ **Going below -128 wraps to +127**

This is **modular arithmetic** with modulus 256.

🌟 PART 2 — How “wrapping” actually happens

We take a simple byte:

Range = [-128, 127]

Total values = 256

If you ADD 1 to the maximum positive number:

Example 1: 127 wraps to -128

0111 1111 (127)

+ 0000 0001

1000 0000 → (-128)

That top bit (1) means the value is **negative**.

Example 2: -128 wraps to 127 when subtracting 1

1000 0000 (-128)

- 0000 0001

0111 1111 → (+127)

It's like a circular number line.

🌟 PART 3 — Why this wrap happens (internal mechanism)

Two's complement uses the rule:

If a result doesn't fit in 8 bits → drop extra bits.

This makes the integer space **cyclic**, like a clock:

... 125, 126, 127, -128, -127, ... -1, 0, 1, 2 ...

Just like 12-hour clocks wrap:

12 → 1 → 2 → ... → 12 → 1 → ...

🌟 PART 4 — Practical Java Examples

1 Adding past max byte value

```
byte b = 127;  
  
b++;    // wraps  
  
System.out.println(b);
```

Output:

-128

2 Subtracting past min value

```
byte b = -128;
```

```
b--;    // wraps  
System.out.println(b);
```

Output:

127

③ Overflow in int works the same way (but 32-bit)

```
int x = Integer.MAX_VALUE;  
System.out.println(x + 1);
```

Output:

-2147483648

★ PART 5 — Visual Overflow Animation (byte)

Let's animate the transition from 126 to -128:

126 → 127 → -128 → -127 → -126

Binary:

0111 1110 (126)

0111 1111 (127)

1000 0000 (-128)

1000 0001 (-127)

1000 0010 (-126)

Notice: after hitting 0111 1111, adding 1 flips all bits to 1000 0000.

★ PART 6 — Why JVM does this (as your textbook says)

Two's complement solves:

- ✓ no negative zero
- ✓ arithmetic is consistent
- ✓ comparisons are simple
- ✓ CPU hardware becomes simpler
- ✓ bitwise ops behave predictably

This is why **all modern CPUs** (Intel, AMD, ARM, RISC-V) use two's complement, and Java naturally follows CPU behavior.

★ PART 7 — Deep Insight: Why 1111 1111 = -1?

Compute negative of 1:

1. invert bits

0000 0001 → 1111 1110

2. add 1

$$1111\ 1110 + 1 = 1111\ 1111$$

So $1111\ 1111 = -1$.

This also explains why decrementing 0 gives:

$$0 \rightarrow -1$$

$$0000\ 0000 - 1 = 1111\ 1111$$

PART 8 — Summary Cheat Sheet

Operation	Result
$127 + 1$	$\rightarrow -128$
$-128 - 1$	$\rightarrow 127$
$0 - 1$	$\rightarrow -1$
$-1 + 1$	$\rightarrow 0$

Adds/subtracts overflow $\rightarrow$ wrap around

The integer universe is a **closed circle**, not a straight line.

✓ Why $\sim x$ Feels Like “Moving Around a Wheel” in Two’s Complement

First, picture all byte values as a circle of 256 numbers:

0 to 255

or in signed form:

$-128 \rightarrow -127 \rightarrow \dots \rightarrow -1 \rightarrow 0 \rightarrow 1 \rightarrow \dots \rightarrow 126 \rightarrow 127$

This circle “wraps around.”

✓ 1. Two’s Complement Circle

In **signed 8-bit**, the numbers are arranged like this:

Binary	Decimal
--------	---------

0000 0000 $\rightarrow$ 0

0000 0001 $\rightarrow$ 1

...

0111 1111 $\rightarrow$ 127

1000 0000 $\rightarrow$ -128

1000 0001 $\rightarrow$ -127

...

1111 1111 $\rightarrow$ -1 $\leftarrow$ This is important

Notice:

- 1111 1111 is **NOT** +255...

- It represents -1 .

That's the magic of two's complement.



2. What does $\sim$ actually do?

It **flips every bit**:

$\sim x$ = bitwise invert of x

Example:

$x = 0000\ 0001$ (1)

$\sim x = 1111\ 1110$ (-2)

But here's the key:

 **In two's complement, flipping all bits is equivalent to negating a number and subtracting 1.**

Mathematically:

$$\sim x = -x - 1$$

Check:

$$x = 1$$

$$-x - 1 = -1 - 1 = -2$$

Matches what we saw.

3. Why does bitwise NOT feel like “moving around the wheel”?

Because complementing bits takes you to the **diametric opposite** point on the number circle.

Imagine the circle of 256 points.

$\sim x$ jumps to the number **128 steps away**, directly opposite.

Example:

~ 5

Binary:

$5 = 0000\ 0101$

$\sim 5 = 1111\ 1010 \rightarrow -6$

Check with formula:

$\sim x = -x - 1 = -5 - 1 = -6$

So:

$5 \text{ -----} \rightarrow -6$

They are on opposite sides of the circle.

Why does this happen?

Because two's complement wraps around:

$\dots \rightarrow -3 \rightarrow -2 \rightarrow -1 \rightarrow 0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \dots$

Flipping bits sends you to the “reflected point” across the center.

4. Why NOT does not give a clean negative like -x?

Because $-(x) = \sim x + 1$.

We add 1 to avoid negative zero.

So instead:

$\sim 0 = 11111111 = -1$

$\sim 1 = 11111110 = -2$

$\sim 2 = 11111101 = -3$

...

$\sim -1 = 00000000 = 0$

This behavior matches the wheel structure.

Practical intuition (the real meaning)

When you do:

$\sim x$

You are not simply flipping bits...

You are jumping to:

The number that is exactly opposite x on the two's-complement number wheel



Summary Table

x	Binary	~x (binary)	Decimal result
0	00000000	11111111	-1
1	00000001	11111110	-2
2	00000010	11111101	-3
-1	11111111	00000000	0
-5	11111011	00000100	4

Always:

$$\sim x = -x - 1$$

That is the “wheel movement.”



1. First, understand the range of a signed 8-bit number

Java's **byte** is **8 bits**, so it has:

$$2^8 = 256 \text{ total values}$$

In **two's complement**, the range is:

-128 to +127 (256 total numbers)

Because:

- **1 bit for sign** (most significant bit, MSB)
- **7 bits for magnitude**

So:

- Smallest = **1000 0000** = -128
- Largest = **0111 1111** = 127

2. Why does -128 have no positive equivalent?

Let's try to compute **+128** in 8-bit form.

Binary of 128 is:

1000 0000 ← this is the same bit pattern as -128!

But in **two's complement**, MSB = 1 means **negative**:

- MSB 0 → positive
- MSB 1 → negative

So this pattern **cannot** represent +128.

There is **no place** to put +128 in 8 bits because:

- Positive numbers only go up to 127 (0111 1111)
- 128 would require a 9th bit

Therefore:

-128 has no +128

3. Why does this happen mathematically?

Two's complement range is determined by:

$$\text{Min} = -2^{(n-1)}$$

$$\text{Max} = 2^{(n-1)} - 1$$

For 8 bits:

$$\text{Min} = -2^7 = -128$$

$$\text{Max} = 2^7 - 1 = 127$$

Notice **one extra negative number**.

That's the entire reason.

The count:

Negative values : 128 numbers (from -128 to -1)

Zero : 1 number (0)

Positive values : 127 numbers (1 to 127)

Total:

$$128 + 1 + 127 = 256$$

So the negative side gets **one more slot** — that slot becomes **-128**.

Thus, -128 is "unpaired."

4. Why can't we have +128 instead of -128?

Because that would require **two zeros**:

- +0 (0000 0000)
- -0 (1000 0000)

But negative zero is mathematically useless and very messy:

- How do you compare?
- How do you sort?
- How do you print?
- How do you decide if `-0 == +0`?

So instead, two's complement **uses the negative zero's pattern (1000 0000) to store -128**, eliminating `-0` completely.

This is why the range shifts left by one on the negative side.

5. Why can't we negate -128?

Check this famous Java behavior:

```
byte x = -128;
```

```
byte y = (byte) -x;
```

```
System.out.println(y);
```

Result:

-128

Why?

Because +128 **does not exist** in an 8-bit signed type.

Mathematically:

$-x = 128$

But 128 overflows and wraps around to:

$128 \bmod 256 = 128 \rightarrow 1000\ 0000 \rightarrow -128$

So you get the original value back.

6. Visual intuition (the number wheel)

+127 +126 ... +1 0 -1 -2 ... -127 -128

↑

↓

|_____|

It's a circle.

The wheel has **an odd number of slots**:

256 total values

But zero occupies **one slot**.

The remaining 255 values must be split into + and −:

127 positives

128 negatives ← one extra

That extra negative is **-128**.

There is no symmetric positive because the circle ends.

7. Final Summary — Why −128 has no +128

- 8-bit signed range is **−128 to +127**
- The “zero slot” takes one value
- Positives get 127 slots
- Negatives get 128 slots
- The extra negative value becomes **−128**
- Bit pattern **1000 0000** cannot represent +128
- Negating −128 overflows and returns −128 again

Therefore:

-128 is unique.

It is the only number in Java's byte type without a positive counterpart.

NEGATIVE VALUES (-128 → -1)

-128 → 10000000

-127 → 10000001

-126 → 10000010

-125 → 10000011

-124 → 10000100

-123 → 10000101

-122 → 10000110

-121 → 10000111

-120 → 10001000

-119 → 10001001

-118 → 10001010

-117 → 10001011

-116 → 10001100

-115 → 10001101

-114 → 10001110

-113 → 10001111

-112 → 10010000

-111 → 10010001

-110 → 10010002

-109 → 10010011

-108 → 10010100

-107 → 10010101

-106 → 10010110

-105 → 10010111

-104 → 10011000

-103 → 10011001

-102 → 10011010

-101 → 10011011

-100 → 10011100

-99 → 10011101

-98 → 10011110

-97 → 10011111

-96 → 10100000

-95 → 10100001

-94 → 10100010

-93 → 10100011

-92 → 10100100

-91 → 10100101

-90 → 10100110

-89 → 10100111

-88 → 10101000

-87 → 10101001

-86 → 10101010

-85 → 10101011

-84 → 10101100

-83 → 10101101

-82 → 10101110

-81 → 10101111

-80 → 10110000

-79 → 10110001

-78 → 10110010

-77 → 10110011

-76 → 10110100

-75 → 10110101

-74 → 10110110

-73 → 10110111

-72 → 10111000

-71 → 10111001

-70 → 10111010

-69 → 10111011

-68 → 10111100

-67 → 10111101

-66 → 10111110

-65 → 10111111

-64 → 11000000

-63 → 11000001

-62 → 11000002

-61 → 11000011

-60 → 11000100

-59 → 11000101

-58 → 11000110

-57 → 11000111

-56 → 11001000

-55 → 11001001

-54 → 11001010

-53 → 11001011

-52 → 11001100

-51 → 11001101

-50 → 11001110

-49 → 11001111

-48 → 11010000

-47 → 11010001

-46 → 11010010

-45 → 11010011

-44 → 11010100

-43 → 11010101

-42 → 11010110

-41 → 11010111

-40 → 11011000

-39 → 11011001

-38 → 11011010

-37 → 11011011

-36 → 11011100

-35 → 11011101

-34 → 11011110

-33 → 11011111

-32 → 11100000

-31 → 11100001

-30 → 11100010

-29 → 11100011

-28 → 11100100

-27 → 11100101

-26 → 11100110

-25 → 11100111
-24 → 11101000
-23 → 11101001
-22 → 11101010
-21 → 11101011
-20 → 11101100
-19 → 11101101
-18 → 11101110
-17 → 11101111
-16 → 11110000
-15 → 11110001
-14 → 11110010
-13 → 11110011
-12 → 11110100
-11 → 11110101
-10 → 11110110
-9 → 11110111
-8 → 11111000
-7 → 11111001
-6 → 11111010
-5 → 11111011
-4 → 11111100
-3 → 11111101
-2 → 11111110
-1 → 11111111

ZERO

0 → 00000000

POSITIVE VALUES (1 → 127)

1 → 00000001

2 → 00000010

3 → 00000011

4 → 00000100

5 → 00000101

6 → 00000110

7 → 00000111

8 → 00001000

9 → 00001001

10 → 00001010

11 → 00001011

12 → 00001100

13 → 00001101

14 → 00001110

15 → 00001111

16 → 00010000

17 → 00010001

18 → 00010010

19 → 00010011

20 → 00010100

...

30 → 00011110

31 → 00011111

...

63 → 00111111

64 → 01000000

...

100 → 01100100

101 → 01100101

...

120 → 01111000

121 → 01111001

122 → 01111010

123 → 01111011

124 → 01111100

125 → 01111101

126 → 01111110

127 → 01111111

✓ Why does two's complement NOT decide "sign bit + remaining bits = value"?

Because two's complement does NOT work like this:

✗ *Sign bit + remaining bits interpreted as magnitude*
(This is how **sign-magnitude** or **1's complement** works — NOT two's complement.)

In **two's complement**, the rule is:

★ ****The entire 8 bits together represent the value using a fixed mathematical formula.**

There is *no splitting*, no "sign + magnitude".**

🧠 THE REAL RULE in Two's Complement

For an 8-bit signed number:

- If **MSB = 0** → positive:
value = normal binary.
- If **MSB = 1** → negative:
value = (binary value) - 256

Because $256 = 2^8$ (the range of an 8-bit register)

⚡ **Example: 11111110**

Binary value = 254
MSB = 1 → negative

So:

value = 254 - 256 = -2

Simple and exact.
No guesswork.
No picking digits.



Your question:

How does the CPU know not to pick only the last 2 digits?

Why not last 3 digits?

Why not split as 110 → 6 and sign = negative = -6?

Because **two's complement NEVER uses "sign + remaining bits"**.

That method belongs to **sign-magnitude**, not two's complement.

In two's complement:

- All 8 bits act as one whole number.
 - The formula defines the meaning.
 - You cannot choose arbitrary splits like "last 2 bits" or "last 3 bits".
-



Let's test your alternate interpretations:

✗ If CPU assumed:

"Last 3 bits = number → 110 = 6, sign bit = negative → -6"

Then:

11111010 should also be -6

11100110 should also be -6

11000010 should also be -6

But they aren't.

Two's complement is **consistent** because it uses **a mathematical rule**, not splitting logic.

★ THE ACTUAL MAPPING (fixed by math)

Let's see why -6 is actually:

6 → 00000110

invert → 11111001

add 1 → 11111010 ← actual -6

And -2 is:

2 → 00000010

invert → 11111101

add 1 → 11111110 ← actual -2

Notice:

- they are **unique**
 - they follow **one formula**
 - no arbitrary splitting is possible
-

Visual Summary

Two's Complement:

Interpret the ENTIRE binary number.

If MSB = 0 → value = binary

If MSB = 1 → value = binary - 256

The left shift operator, `<<`, shifts all of the bits in a value to the left a specified number of times. It has this general form: `value << num`. Here, `num` specifies the number of positions to left-shift the value in `value`. That is, the `<<` moves all of the bits in the specified value to the left by the number of bit positions specified by `num`. For each shift left, the high-order bit is shifted out (and lost), and a zero is brought in on the right. This means that when a left shift is applied to an `int` operand, bits are lost once they are shifted past bit position 31. If the operand is a `long`, then bits are lost after bit position 63. Java's automatic type promotions produce unexpected results when you are shifting byte and short values. As you know, byte and short values are promoted to `int` when an expression is evaluated. Furthermore, the result of such an expression is also an `int`. This means that the outcome of a left shift on a byte or short value will be an `int`, and the bits shifted left will not be lost until they shift past bit position 31. Furthermore, a negative byte or short value will be sign-extended when it is promoted to `int`. Thus, the high-order bits will be filled with 1's. For these reasons, to perform a left shift on a byte or short implies that you must discard the high-order bytes of the `int` result. For example, if you left-shift a byte value, that value will first be promoted to `int` and then shifted. This means that you must discard the top three bytes of the result if what you want is the result of a shifted byte value. The easiest way to do this is to simply cast the result back into a byte.

The right shift operator, `>>`, shifts all of the bits in a value to the right a specified number of times. Its general form is: `value >> num`. Here, `num` specifies the number of positions to right-shift the value in `value`. That is, the `>>` moves all of the bits in the specified value to the right the number of bit positions specified by `num`.

First Rule: Why byte/short Behave Weird

In Java:

- `byte` = 8 bits

- `short` = 16 bits
- `int` = 32 bits

BUT...

👉 Whenever you do arithmetic or bitwise operations on `byte` or `short`, Java **promotes them to int first**.

This is called **automatic type promotion**.



Example 1 — Simple Byte Left Shift

```
byte b = 5;  
int result = b << 1;  
System.out.println(result);
```

Step 1: Write 5 in binary (byte = 8 bits)

00000101 (5)

Step 2: Promotion to int (32 bits)

Java converts it to:

00000000 00000000 00000000 00000101

Step 3: Shift Left by 1

00000000 00000000 00000000 00001010

This equals 10.

So output is:

10

So far everything looks normal.

Now The REAL Problem — Negative Byte

This is where it gets interesting.

```
byte b = -5;  
int result = b << 1;  
System.out.println(result);
```

Step 1: Represent -5 in byte (8 bits)

First write +5:

00000101

Now take 2's complement:

Invert:

11111010

Add 1:

11111011

So:

11111011 (-5 as byte)

Step 2: Promotion to int (Sign Extension)

Because it's negative, Java fills the left bits with 1s.

It becomes:

11111111 11111111 11111111 11111011

Notice how the 1s extend.

This is called **sign extension**.

Step 3: Shift Left by 1

11111111 11111111 11111111 11110110

Now convert back to decimal:

This equals **-10**

So output:

-10



Now Here's The Big Problem

If you write:

```
byte b = 5;  
byte result = b << 1; // ❌ Compile error
```

Why?

Because:

- `b << 1` becomes an **int**
 - You're trying to store int into byte
 - Java says: possible lossy conversion
-



Correct Way

```
byte b = 5;  
byte result = (byte) (b << 1);
```

Now it compiles.

Because you're explicitly saying:

Yes, I know I may lose upper bits.

The “Losing Upper Bits” Problem

Let's see real overflow.

```
byte b = 120;  
byte result = (byte)(b << 1);  
System.out.println(result);
```

Step 1: 120 in binary

01111000

Promoted to int:

00000000 00000000 00000000 01111000

Step 2: Shift left by 1

00000000 00000000 00000000 11110000

This equals 240.

Step 3: Cast back to byte (8 bits only)

Keep only last 8 bits:

11110000

This is negative in byte.

Find decimal:

Invert:

00001111

Add 1:

00010000 = 16

So result = **-16**



Output

-16

Even though mathematically $120 \times 2 = 240$.

Because byte max = 127.
It overflowed.



Why The Book Says You Must Discard High Bytes

When shifting a byte:

1. It becomes 32-bit int
2. Shift happens on 32 bits
3. If you want the result as byte,
you must remove the upper 24 bits

Casting does that.



Clean Mental Model

When doing:

```
byte b = 10;  
b << 1;
```

Think:

(byte → int → shift → int result)

Never:

(byte → shift → byte)

Java never shifts in 8-bit mode.



Summary Table

Step	What Happens
byte used in expression	Promoted to int
Negative byte	Sign-extended
Shift	Happens on 32 bits
Assign back to byte	Must cast
Overflow	Upper bits discarded

◆ **1** First: What Does **>>** Do?

>> is called **arithmetic right shift**.

It:

- Shifts bits to the right
- Fills the leftmost (high-order) bit with the **previous sign bit**

This is called **sign extension**.

It preserves the sign of the number.

Example 1: Positive Number with >>

```
int x = 8;
```

```
System.out.println(x >> 1);
```

Binary of 8:

```
00000000 00000000 00000000 00001000
```

Shift right by 1:

```
00000000 00000000 00000000 00000100
```

Result = 4 

Since sign bit was 0, 0 keeps filling in.

Example 2: Negative Number with >>

```
int x = -8;
```

```
System.out.println(x >> 1);
```

Binary of -8 (32-bit):

```
11111111 11111111 11111111 11111000
```

Now shift right by 1 using >>:

```
11111111 11111111 11111111 11111100
```

Notice the left side is filled with 1's (because it was negative).

This equals:

-4

So:

$-8 \gg 1 = -4$

Sign preserved.

◆ 2 Now What Is $\ggg$?

$\ggg$ is called **logical right shift** (unsigned right shift).

It:

- Shifts bits to the right
 - ALWAYS fills left side with 0
 - Does NOT preserve sign
-

🔥 Example 3: Negative Number with

$\ggg$

```
int x = -8;
```

```
System.out.println(x >>> 1);
```

Binary of -8:

11111111 11111111 11111111 11111000

Shift right by 1 using `>>>`:

01111111 11111111 11111111 11111100

Notice:

Leftmost bit became 0.

Now this is a HUGE positive number.

Let's calculate:

This equals:

2147483644

😲 Big difference!

Side-by-Side Comparison

Expressio n	Result
----------------	--------

<code>-8 >> 1</code>	-4
----------------------------	----

<code>-8 >>> 1</code>	214748364 4
--------------------------------	----------------

Why This Is Useful (Graphics Example)

Imagine pixel color stored in int:

ARGB format:

Alpha (8 bits)

Red (8 bits)

Green (8 bits)

Blue (8 bits)

Example:

11111111 10101010 11001100 11110000

If you use `>>` to extract color components and the number is negative, Java will sign-extend and mess up your result.

So in graphics we use:

`>>>`

Because we don't care about sign.
We just want raw bits.

◆ **3 Important Rule**

- `>>` = Signed shift (preserves sign)
- `>>>` = Unsigned shift (fills with zero)

Special Note

There is NO `<<<` in Java.

Only:

`<<`

>>

>>>



Visual Summary

Right shift with sign preservation:

1110 0000 >> 1

1111 0000

Unsigned shift:

1110 0000 >>> 1

0111 0000



Mental Model

Think:

- >> → "Keep number negative if it was negative"
- >>> → "I don't care about sign, just shift bits"



Final Deep Insight

Java has no unsigned int type.

So >>> is how Java lets you simulate unsigned behavior.

Relational Operators

The *relational operators* determine the relationship that one operand has to the other. Specifically, they determine equality and ordering. The relational operators are shown here:

Operator	Result
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

The outcome of these operations is a **boolean** value. The relational operators are most frequently used in the expressions that control the **if** statement and the various loop statements.

No, Java bitwise AND (`&`) and logical AND (`&&`) are not the same; they differ in their operands, operation, and a crucial behavior called "short-circuiting".

Key Differences

Feature	Bitwise AND (<code>&</code>)	Logical AND (<code>&&</code>)
Operates On	Integer types (byte, short, int, long, char) and boolean types	Only boolean types
Operation	Performs an AND operation on each corresponding bit of the operands	Performs a boolean logical operation on the truth values of the operands
Short-Circuiting	No (evaluates both operands always)	Yes (stops evaluating if the first operand is false)
Return Type	An integer type (if used with integers) or boolean (if used with booleans)	A boolean value

in order to function properly. For example, the following code fragment shows how you can take advantage of short-circuit logical evaluation to be sure that a division operation will be valid before evaluating it:

```
if (denom != 0 && num / denom > 10)
```

Since the short-circuit form of AND (**&&**) is used, there is no risk of causing a run-time exception when **denom** is zero. If this line of code were written using the single **&** version of AND, both sides would be evaluated, causing a run-time exception when **denom** is zero.

It is standard practice to use the short-circuit forms of AND and OR in cases involving Boolean logic, leaving the single-character versions exclusively for bitwise operations. However, there are exceptions to this rule. For example, consider the following statement:

```
if (c==1 & e++ < 100) d = 100;
```

Here, using a single **&** ensures that the increment operation will be applied to **e** whether **c** is equal to 1 or not.

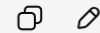
Highest						
++ (postfix)	-- (postfix)					
++ (prefix)	-- (prefix)	~	!	+(unary)	-(unary)	(type-cast)
*	/	%				
+	-					
>>	>>>	<<				
>	>=	<	<=	instanceof		
==	!=					
&						
^						
&&						
?:						
->						
=	op=					
Lowest						

Table 4-1 The Precedence of the Java Operators

🔥 2 Why does this compile?

Java

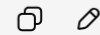
```
byte b = 10;  
b += 20;
```



But this does NOT?

Java

```
byte b = 10;  
b = b + 20;
```



The statement `b += 20;` compiles successfully because the compound assignment operator (`+=`) in Java includes an implicit casting mechanism, whereas `b = b + 20;` fails because standard arithmetic operations (`+`) in Java do not have this automatic casting, potentially leading to a loss of precision.

Here is a detailed explanation at the compiler level:

The failing statement: `b = b + 20;`

This statement fails due to type incompatibility.

1. **Implicit Type Promotion:** In Java, arithmetic operations on `byte`, `short`, or `char` types automatically promote the operands to `int` (or `long` if any operand is a `long`) before the operation is performed. This is known as binary numeric promotion.
2. **The Operation:** The expression `b + 20` involves a `byte` (`b`) and an `int` (`20`). Both are promoted to `int`, and the result of the addition is an `int`.
3. **The Assignment Issue:** The compiler then attempts to assign this `int` result back into a `byte` variable (`b`).
4. **Compiler Error:** The compiler flags this as an error ("incompatible types: possible lossy conversion from `int` to `byte`") because a 32-bit `int` value might be too large to fit into an 8-bit `byte` variable, leading to a potential loss of data.

The compiling statement: `b += 20;`

This statement compiles because the Java Language Specification (JLS) explicitly defines compound assignment operators to handle this type conversion automatically.

1. The JLS Rule: The expression `E1 op= E2` is defined to be equivalent to `E1 = (T) ((E1) op (E2))`, where `T` is the type of `E1`.
2. The Implicit Cast: In your case, `b += 20;` is internally treated by the compiler as `b = (byte) (b + 20);`.
3. The Result: The operation `b + 20` still results in an `int`, but the compiler-inserted explicit cast `(byte)` converts the `int` result back into a `byte` type, which can then be assigned to the variable `b` without a compiler error.

In summary, the compound operator is a syntactic shortcut that includes a built-in explicit cast, which the simple assignment operator lacks

Why does Java NOT allow operator overloading (except `+` for String)?

Java's design eschews general operator overloading primarily to maintain simplicity, readability, and consistency within the language. The designers aimed to prevent the common problems associated with operator overloading in other languages, such as C++, which often led to confusion and complexity.

The primary motivations for this decision were to prevent the following problems:

- Complexity and Readability: The most significant issue with operator overloading is that the same symbol (e.g., `+`, `-`, `*`) can have vastly different meanings depending on the class. For a developer reading code written by someone else, this introduces ambiguity; they would have to constantly check the definition of a class to understand what an operator is actually doing. Java designers wanted code to be easily readable and understandable without this extra effort.

- Preventing "Clever" but Confusing Code: Operator overloading enables developers to write "clever" implementations (e.g., using the `*` operator to represent matrix multiplication, or the `<<` operator for output streams), which can be intuitive for the author but completely non-obvious for others. This "syntactic sugar" often becomes "syntactic salt", hindering maintainability and collaboration.
- Language Consistency: By restricting operations to methods with explicit names (like `add()`, `subtract()`, etc.), Java ensures a consistent way to perform operations across different types. Developers always know where to look for an operation's implementation.
- Simpler Parser and Tooling: Banning operator overloading makes the language specification simpler, which in turn makes the compiler and development tools easier to implement.
- Avoiding Abuse: In languages like C++, operator overloading could be abused in ways that violate mathematical intuition (e.g., overloading `==` to mean something other than equality comparison). Java avoids this potential for misuse by forcing developers to use descriptive method names.

The one exception, the `+` operator for `String` concatenation, was included because string manipulation is a fundamental and frequently used operation in most programs, and the `+` syntax is universally intuitive for this purpose. The designers considered it a necessary convenience that did not compromise the language's core principles of readability and simplicity.

What happens internally when you write:

```
int x = Integer.MAX_VALUE + 1;
```

Why does Java not throw overflow exceptions for primitives

When you write `int x = Integer.MAX_VALUE + 1;` in Java, integer overflow occurs, and `x` will hold the value `Integer.MIN_VALUE`. This happens because Java's primitive

integer types use two's complement representation and wrap around on overflow, rather than throwing an exception.

What happens internally

1. Binary Representation: `Integer.MAX_VALUE` is represented in a 32-bit signed integer as `01111111 11111111 11111111 11111111`.
2. Addition: When you add 1 to this value, the internal binary arithmetic proceeds as follows:
 1. The least significant bit (LSB) becomes 0, and a carry is generated.
 2. This carry propagates across all the bits.
 3. Eventually, the most significant bit (MSB) or sign bit flips from 0 to 1.
3. Resulting Value: The resulting binary representation is `10000000 00000000 00000000 00000000`. In two's complement, this bit pattern represents `Integer.MIN_VALUE`.
4. Assignment: This resulting `Integer.MIN_VALUE` is then assigned to the variable `x`. The actual value of `x` becomes -2,147,483,648.

Why Java does not throw overflow exceptions for primitives

Java was intentionally designed not to throw exceptions for arithmetic overflows on primitive types for several reasons:

- Performance: Checking for overflow at every arithmetic operation would introduce significant overhead, impacting the performance of numerical computations. This was a key design consideration for making Java fast.
- C/C++ Heritage: Java's design draws inspiration from C and C++, where integer overflow also wraps around silently without exceptions. This design choice maintained consistency with established conventions that many early Java developers were familiar with.
- Explicit Handling: Java designers decided to leave overflow handling to the programmer when necessary.
 - You can use larger data types like `long` to store the result of operations that might exceed the `int` range.

- You can use the `Math` class methods like `Math.addExact()`, `Math.subtractExact()`, etc., which *do* perform overflow checks and throw an `ArithmeticException` if an overflow occurs.
- Java 8 introduced a new `StrictMath` API for exact arithmetic operations.

This design philosophy means that developers must be aware of potential overflows and handle them explicitly if silent wrapping behavior is not desired.

Why does this fail?

```
int i = 0;
```

```
i = i++;
```

The code `int i = 0; i = i++;` does not increment `i` because it uses a post-increment operation. This is followed by an assignment that overwrites the incremented value with the original value. The expression `i++` gets the current value (0) and increments the local variable to 1. However, it returns the old value (0), which is assigned back to `i`. This results in `i` remaining 0.

Bytecode Analysis

Assuming `i` is stored in the first local variable slot:

1. 0 is pushed onto the operand stack.
2. 0 is stored in variable `i`. (`i=0`)
3. Variable `i` (0) is loaded onto the operand stack.
4. Local variable `i` is incremented in memory by 1. (`i` is now 1).
5. The value currently on the stack (0) is stored back into `i`. (`i` is now 0 again).

The post-increment (`i++`) makes the value on the stack the value before the increment. The assignment (`i = ...`) overwrites the incremented value (`iinc`) with the stale value.


```
8
9 public class Main
10 {
11     public static void main(String[] args) {
12         int i = 0;
13         i = i++;
14         System.out.println(i);
15     }
16 }
```



0

...Program finished with exit code 0
Press ENTER to exit console.□